MCMP117 · WEEK 10

Fundamentals of Operating Systems

Week 10 · File & Directory Operations

Create, copy, move, delete, view and link files — and know what the kernel does each time.

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 9 in Four Ideas

Terminal, shell, kernel

The shell (bash) is a program that reads your command, starts other programs, and they ask the kernel for work through system calls (Week 2).

Command anatomy

command -options arguments

ls -l /etc

Options change how; arguments say what.

Paths

Absolute: /home/student/lab

Relative: docs, ../data

Shortcuts: ~ . .. - (cd - goes back)

Tree and help

/home /etc /usr /var /tmp /dev /proc (FHS). Help: --help, man, Tab completion, history and !!.

Last week we walked around the tree with pwd, ls and cd. Today we change it: every command today creates, moves or removes a name, an inode or data.

Today's Agenda

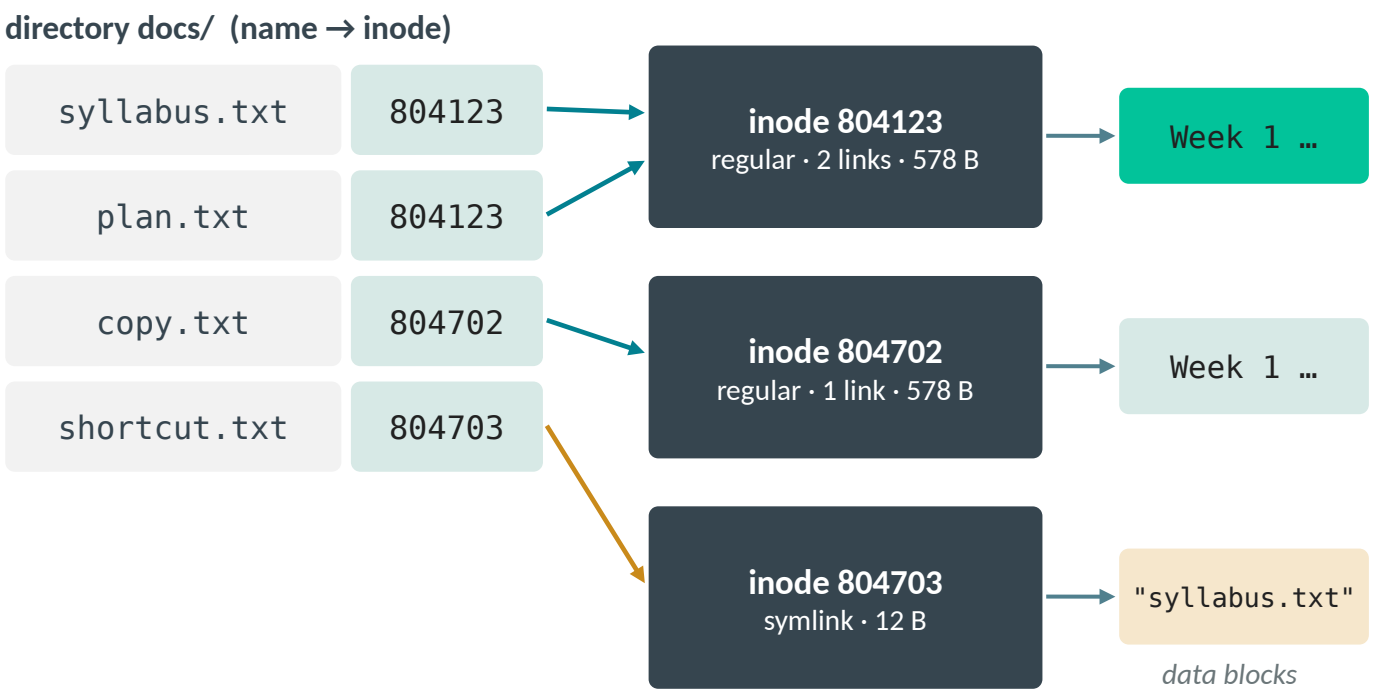
1. **Names, inodes, system calls** What really changes on disk when you work with files (Week 7 recap).
2. **Create, copy, move, delete** mkdir, touch, cp, mv, rm, rmdir — and why rm has no undo.
3. **Wildcards and braces** * ? [ ] { } — and who expands them: the shell, not the command.
4. **Names with spaces** Quoting with " ", ' ' and \.
5. **Looking inside files** cat, less, head, tail -f, wc, file, stat, du, df.
6. **Links and the lab** ln and ln -s again; then 35 minutes of graded hands-on exercises.

PART 1

Names, Inodes and System Calls

Week 7 theory, seen from the command line.

What Really Changes: Names, Inodes, Data



Command	What changes
`touch new`	new name + new (empty) inode
`cp a b`	new name + new inode + copy of the data
`mv a b`	only the name; same inode, data untouched
`ln a b`	new name → same inode; link count + 1
`ln -s a b`	new name + small inode storing the path a
`rm a`	name removed; link count − 1; data freed at 0

Real inode numbers from ls -li (demo_links.sh): plan.txt is a hard link, copy.txt a copy, shortcut.txt a symbolic link that stores a path.

Each Command Is a Program That Makes a System Call

```
student@okan-lab:~/lab$ strace -e trace=mkdir,mkdirat mkdir newdir
mkdir("newdir", 0777)                   = 0
+++ exited with 0 +++
student@okan-lab:~/lab$ strace -e trace=rename,renameat,renameat2 mv newdir olddir
renameat2(AT_FDCWD, "newdir", AT_FDCWD, "olddir", RENAME_NOREPLACE) = 0
+++ exited with 0 +++
student@okan-lab:~/lab$ touch file1
student@okan-lab:~/lab$ strace -e trace=link,linkat ln file1 file2
linkat(AT_FDCWD, "file1", AT_FDCWD, "file2", 0) = 0
+++ exited with 0 +++
student@okan-lab:~/lab$ strace -e trace=symlink,symlinkat ln -s olddir sym
symlinkat("olddir", AT_FDCWD, "sym")    = 0
+++ exited with 0 +++
student@okan-lab:~/lab$ strace -e trace=unlink,unlinkat rm file1 file2 sym
unlinkat(AT_FDCWD, "file1", 0)          = 0
unlinkat(AT_FDCWD, "file2", 0)          = 0
unlinkat(AT_FDCWD, "sym", 0)            = 0
+++ exited with 0 +++
student@okan-lab:~/lab$ strace -e trace=rmdir,unlinkat rmdir olddir
rmdir("olddir")                         = 0
+++ exited with 0 +++
```

One command, one call

- **mkdir** → mkdir()
- **mv** → renameat2()
- **ln** → linkat()
- **ln -s** → symlinkat()
- **rm** → unlinkat()
- **rmdir** → rmdir()
- The kernel calls deleting "unlink": it removes one name (link) of an inode.

PART 2

Create, Copy, Move, Delete

mkdir · touch · cp · mv · rm · rmdir

LIVE DEMO

mkdir: Make Directories (-p for Whole Paths)

```
student@okan-lab:~/lab$ mkdir test
student@okan-lab:~/lab$ ls -ld test
drwxrwxr-x 2 student student 4096 Oct 12 08:47 test
student@okan-lab:~/lab$ mkdir test
student@okan-lab:~/lab$ mkdir trip/day1
mkdir: cannot create directory ‘trip/day1’: No such file or directory
student@okan-lab:~/lab$ mkdir -p trip/day1
student@okan-lab:~/lab$ mkdir -p work/2026/october
student@okan-lab:~/lab$ tree work trip
work
└── 2026
    └── october
trip
└── day1

5 directories, 0 files
```

What to observe

- mkdir name creates one empty directory.
- **File exists:** the name is taken.
- **No such file or directory:** a parent (trip) is missing.
- **-p:** creates every missing parent and is silent if the directory already exists.
- ls -ld shows the directory itself, not its contents.

LIVE DEMO

touch: Create Empty Files, Change Timestamps

```
student@okan-lab:~/lab$ touch todo.txt
student@okan-lab:~/lab$ ls -l todo.txt
-rw-rw-r-- 1 student student 0 Oct 12 08:47 todo.txt
student@okan-lab:~/lab$ touch -d "2026-01-15 09:30" todo.txt
student@okan-lab:~/lab$ ls -l todo.txt
-rw-rw-r-- 1 student student 0 Jan 15  2026 todo.txt
student@okan-lab:~/lab$ stat todo.txt
  File: todo.txt
  Size: 0               Blocks: 0          IO Block: 4096   regular empty file
Device: 254,0   Inode: 804708      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1010/ student)   Gid: ( 1012/ student)
Access: 2026-01-15 09:30:00.000000000 +0000
Modify: 2026-01-15 09:30:00.000000000 +0000
Change: 2026-10-12 08:47:56.442757013 +0000
 Birth: 2026-10-12 08:47:56.430757013 +0000
```

What to observe

- **No such file:** touch creates it, 0 bytes.
- **File exists:** touch only sets its times to now; the content is not changed.
- **-d "date":** set a chosen time.
- ls -l shows the year instead of the time for dates more than 6 months away.
- stat shows all the times kept in the inode.

Three Timestamps: Which Action Changes Which?

Action (example)	atime (Access)	mtime (Modify)	ctime (Change)
read the data: cat f	yes (updated lazily)	—	—
write the data: echo x >> f	—	yes	yes
touch f	yes	yes	yes
rename: mv f g	—	—	yes
add a name: ln f h	—	—	yes
change permissions: chmod (Week 11)	—	—	yes

ctime is NOT the creation time: it is the last change of the inode. Creation is "Birth". ls -l shows mtime; cp -p keeps the original mtime.

LIVE DEMO

cp: Copy Files

```
student@okan-lab:~/lab$ cp docs/syllabus.txt backup/
student@okan-lab:~/lab$ cp docs/syllabus.txt backup/syllabus_copy.txt
student@okan-lab:~/lab$ ls backup
syllabus.txt  syllabus_copy.txt
student@okan-lab:~/lab$ cp -v docs/notes_01.txt docs/notes_02.txt backup/
'docs/notes_01.txt' -> 'backup/notes_01.txt'
'docs/notes_02.txt' -> 'backup/notes_02.txt'
student@okan-lab:~/lab$ cp -i docs/syllabus.txt backup/
cp: overwrite 'backup/syllabus.txt'? n
student@okan-lab:~/lab$ cp projects/alpha backup/
cp: -r not specified; omitting directory 'projects/alpha'
```

What to observe

- **cp SOURCE TARGET:** a target directory keeps the name; otherwise it is the new name.
- **Several sources:** the last argument must be a directory.
- **-v:** say what is done. -i: ask before overwriting.
- **Warning:** without -i, cp overwrites an existing file silently.
- Directories need -r.

LIVE DEMO

cp -r: Copy a Whole Directory Tree

```
student@okan-lab:~/lab$ cp -r projects/alpha backup/
student@okan-lab:~/lab$ tree backup
backup
├── alpha
│   ├── README.md
│   ├── docs
│   │   └── design.txt
│   └── src
│       └── main.c
├── notes_01.txt
├── notes_02.txt
├── syllabus.txt
└── syllabus_copy.txt

4 directories, 7 files
student@okan-lab:~/lab$ ls -i docs/syllabus.txt backup/syllabus.txt
804330 backup/syllabus.txt  804123 docs/syllabus.txt
```

What to observe

- **-r (recursive):** the directory and everything below it.
- **A copy is a new file:** new inode number, its own data blocks.
- Change the copy: the original is untouched (and the other way round).
- **cp -p:** also keep times and permissions; cp -a for exact backups.

LIVE DEMO

mv: Rename and Move (Same Inode!)

```
student@okan-lab:~/lab$ mv backup/syllabus.txt backup/syllabus_old.txt
student@okan-lab:~/lab$ ls -i backup/syllabus_old.txt
804330 backup/syllabus_old.txt
student@okan-lab:~/lab$ mv -v docs/notes_03.txt backup/
renamed 'docs/notes_03.txt' -> 'backup/notes_03.txt'
student@okan-lab:~/lab$ mv backup/alpha backup/alpha_2026
student@okan-lab:~/lab$ ls docs backup
backup:
alpha_2026    notes_02.txt  syllabus_copy.txt
notes_01.txt  notes_03.txt  syllabus_old.txt

docs:
'final report.txt'   notes_01.txt   syllabus.txt
 linux_quotes.txt    notes_02.txt
```

What to observe

- **Rename:** mv old new in the same directory.
- **Move:** mv file dir/ — the name moves to another directory.
- **Same inode** (804330) before and after: only the directory entry changed.
- Directories need no -r. Like cp, mv overwrites silently: use -i.

LIVE DEMO

rm and rmdir: Delete Files and Directories

```
student@okan-lab:~/lab$ rm junk1.txt
student@okan-lab:~/lab$ rm -i junk2.txt
rm: remove regular empty file 'junk2.txt'? y
student@okan-lab:~/lab$ rm -v junk3.txt
removed 'junk3.txt'
student@okan-lab:~/lab$ ls junk*
ls: cannot access 'junk*': No such file or directory
student@okan-lab:~/lab$ rmdir empty_dir
student@okan-lab:~/lab$ rmdir projects/beta
rmdir: failed to remove 'projects/beta': Directory not empty
student@okan-lab:~/lab$ rm projects/beta
rm: cannot remove 'projects/beta': Is a directory
student@okan-lab:~/lab$ rm -rv beta_copy
removed 'beta_copy/src/main.c'
removed directory 'beta_copy/src'
removed 'beta_copy/README.md'
removed 'beta_copy/docs/design.txt'
removed directory 'beta_copy/docs'
removed directory 'beta_copy'
```

What to observe

- rm file: no question, no message, no recycle bin.
- **-i:** ask first. -v: report each removal.
- **rmdir:** only EMPTY directories — a safe tool.
- **rm -r dir:** the directory and everything inside it, recursively.
- **Warning:** -f (force) never asks and hides errors.

SAFETY

rm Is Forever: Delete Safely

No recycle bin

rm calls unlinkat(). At link count 0 the blocks are freed and soon reused. The desktop Trash is a feature of the file manager, not of the kernel.

Look before you delete

Run the same pattern with ls (or echo) first:

ls *.tmp → rm *.tmp

Check where you are with pwd.

Ask when unsure

rm -i, rm -ri, cp -i, mv -i ask before each removal or overwrite. Answer y or n.

One space too many

rm -r ~/lab /old (typo: a space)

deletes ~/lab AND tries /old.

rm -rf never asks — and with sudo it can destroy the system.

Safe habit: pwd → ls the pattern → rm. Backups are the only real undo.

CHEAT SHEET 1

Creating, Copying, Moving, Deleting

Command	What it does	Useful options
mkdir dir	create a directory	-p whole path, no error if it exists
touch file	create an empty file / update its times	-d "2026-01-15 09:30" set a time
cp src dst	copy files (new inode)	-r directories -i ask -v verbose -p keep times
mv src dst	rename or move (same inode)	-i ask before overwriting -v verbose
rm file	remove names — no undo	-i ask -r recursive -v verbose -f force (!)
rmdir dir	remove an empty directory	-p also empty parents

Pattern of all four: command -options sources… target. With several sources, the target must be an existing directory.

INTERMISSION

10-Minute Break

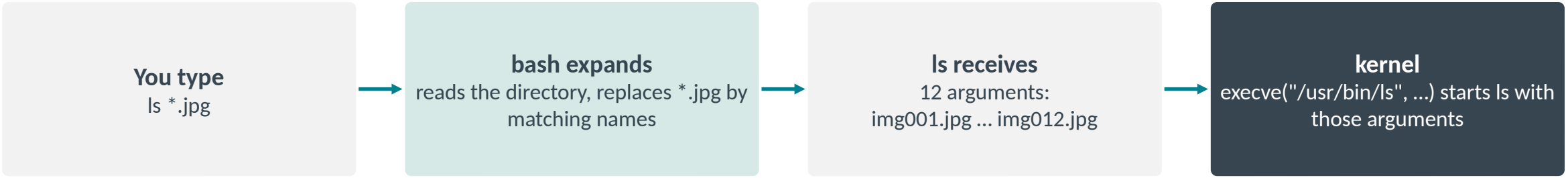
Next: wildcards and braces, names with spaces, looking inside files, links — and the lab.

PART 3

Wildcards, Braces and Quoting

Working with many files at once — and with awkward names.

Who Expands *? The Shell, Not the Command




```
student@okan-lab:~/lab/photos$ echo *.png
img013.png
student@okan-lab:~/lab/photos$ echo *.gif
*.gif
student@okan-lab:~/lab$ ls junk*
ls: cannot access 'junk*': No such file or directory
```

Link to Week 3: bash expands, then fork() + exec() start ls with the finished list of arguments.

Consequences

- echo shows what a command will receive.
- Every command "supports" wildcards — it never sees them.
- No match: bash leaves the pattern as it is, so ls looks for a file literally named junk*.
- Patterns match names, not contents; * skips hidden names (starting with .).

Wildcard Patterns (in ~/lab/photos)

Pattern	Matches	Example	Result (real)
*	any string, also the empty one	*.jpg	12 names: img001.jpg … img012.jpg
?	exactly one character	img00?.jpg	9 names: img001.jpg … img009.jpg
[0-2]	one character from a range	img01[0-2].jpg	img010.jpg img011.jpg img012.jpg
[135]	one of the listed characters	img00[135].jpg	img001.jpg img003.jpg img005.jpg
[jJ]	either case of a letter	*.[jJ][pP]*	the 12 .jpg, holiday.jpeg, IMG_014.JPG
[!0-9]	one character NOT in the set	img0[!0]*	img010.jpg … img013.png

Linux names are case-sensitive: *.jpg does not match IMG_014.JPG. A leading . (hidden name) must be typed explicitly: .*

LIVE DEMO

Wildcards in Action

```
student@okan-lab:~/lab$ cd photos
student@okan-lab:~/lab/photos$ ls
IMG_014.JPG   img002.jpg  img005.jpg  img008.jpg  img011.jpg
holiday.jpeg  img003.jpg  img006.jpg  img009.jpg  img012.jpg
img001.jpg    img004.jpg  img007.jpg  img010.jpg  img013.png
student@okan-lab:~/lab/photos$ ls *.jpg
img001.jpg  img003.jpg  img005.jpg  img007.jpg  img009.jpg  img011.jpg
img002.jpg  img004.jpg  img006.jpg  img008.jpg  img010.jpg  img012.jpg
student@okan-lab:~/lab/photos$ ls img00?.jpg
img001.jpg  img003.jpg  img005.jpg  img007.jpg  img009.jpg
img002.jpg  img004.jpg  img006.jpg  img008.jpg
student@okan-lab:~/lab/photos$ ls img01[0-2].jpg
img010.jpg  img011.jpg  img012.jpg
student@okan-lab:~/lab/photos$ ls img00[135].jpg
img001.jpg  img003.jpg  img005.jpg
student@okan-lab:~/lab/photos$ ls *.[jJ][pP]*
IMG_014.JPG   img002.jpg  img005.jpg  img008.jpg  img011.jpg
holiday.jpeg  img003.jpg  img006.jpg  img009.jpg  img012.jpg
img001.jpg    img004.jpg  img007.jpg  img010.jpg
```

What to observe

- ls in a terminal prints columns, sorted by name.
- With LC_ALL=C.UTF-8, capitals sort first: IMG_014.JPG is at the top.
- img00? matches img001 … img009 but not img010.
- Always test a pattern with ls before using it with rm or mv.

LIVE DEMO

No Match, Quotes and Brace Expansion

```
student@okan-lab:~/lab/photos$ echo *.gif
*.gif
student@okan-lab:~/lab/photos$ echo "*.jpg"
*.jpg
student@okan-lab:~/lab/photos$ echo {1..5}
1 2 3 4 5
student@okan-lab:~/lab/photos$ echo img{01..03}.jpg
img01.jpg img02.jpg img03.jpg
student@okan-lab:~/lab/photos$ echo report_{draft,final}.txt
report_draft.txt report_final.txt
student@okan-lab:~/lab/photos$ mkdir -p ../trip/{day1,day2,day3}/{photos,notes}
student@okan-lab:~/lab/photos$ tree ../trip
../trip
├── day1
│   ├── notes
│   └── photos
├── day2
│   ├── notes
│   └── photos
└── day3
    ├── notes
    └── photos

10 directories, 0 files
```

What to observe

- **Quotes** switch expansion off.
- **{a,b}:** a list. {1..5}: a range.
- **{01..03}:** zero-padded numbers.
- **Product:** {day1,day2,day3}/{photos,notes} gives 3 × 2 = 6 directories; with trip and the 3 days, tree counts 10.

TWO EXPANSIONS

Wildcards vs. Braces

Wildcards * ? [ ]

- Match names that already exist in a directory.
- Results are sorted alphabetically.
- No match: the pattern stays unchanged.
- Typical use: ls, cp, mv, rm on many existing files.
- Example: mv *.png png/

Braces { , } {1..5}

- Generate strings — the files need not exist.
- Order is kept exactly as written.
- Several braces multiply: {a,b}{1,2} → a1 a2 b1 b2.
- Typical use: mkdir, touch, creating many new names.
- Example: mkdir -p week{01..14}

Both are done by bash before the command starts — and both are switched off by quotes ('…' or "…").

LIVE DEMO

Names with Spaces: Quote or Escape

```
student@okan-lab:~/lab$ ls docs
'final report.txt'   notes_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ cat docs/final report.txt
cat: docs/final: No such file or directory
cat: report.txt: No such file or directory
student@okan-lab:~/lab$ cat "docs/final report.txt"
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ cat 'docs/final report.txt'
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ cat docs/final\ report.txt
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ mkdir my photos
mkdir: cannot create directory ‘photos’: File exists
student@okan-lab:~/lab$ mkdir "my photos"
student@okan-lab:~/lab$ ls -d my*
 my  'my photos'
```

What to observe

- A space separates arguments: cat got TWO names.
- **"..." or '...':** one argument.
- **\ (backslash):** escapes the next character.
- Tab completion inserts the backslash for you.
- mkdir my photos made "my" and failed on "photos".
- **Tip:** use _ or - instead of spaces in your own names.

PART 4

Looking Inside Files

cat · less · head · tail · wc · file · stat · du · df

LIVE DEMO

cat, head and tail

```
student@okan-lab:~/lab$ cat docs/linux_quotes.txt
Talk is cheap. Show me the code.
Given enough eyeballs, all bugs are shallow.
Most good programmers do programming not because they expect to get paid.
Theory and practice sometimes clash. And when that happens, theory loses.
Intelligence is the ability to avoid doing work, yet getting the work done.
student@okan-lab:~/lab$ cat -n docs/notes_01.txt docs/notes_02.txt
     1  Meeting notes 1
     2  Meeting notes 2
student@okan-lab:~/lab$ head -n 3 data/students.csv
id,name,department,midterm,final
2026001,Elif Yilmaz,ECT,96,99
2026002,Zeynep Dogan,ME,95,36
student@okan-lab:~/lab$ tail -n 3 logs/system.log
2026-10-12 14:27:00 INFO  user login
2026-10-12 14:38:00 INFO  backup finished
2026-10-12 14:49:00 INFO  network link up
```

What to observe

- **cat:** print whole files; with several files, one after another (concatenate).
- **-n:** number the lines.
- **head / tail -n N:** first / last N lines (default 10).
- For long files cat scrolls away: use less.

LIVE DEMO

Following a Log with tail -f; Paging with less

Terminal 1

```
student@okan-lab:~/lab$ tail -n 2 -f logs/system.log
2026-10-12 14:38:00 INFO  backup finished
2026-10-12 14:49:00 INFO  network link up
2026-10-12 15:00:00 WARN  disk almost full
^C
```

Terminal 2 (meanwhile, a program appends a line)

```
student@okan-lab:~/lab$ echo "2026-10-12 15:00:00 WARN  disk almost full" >> logs/system.log
```

tail -f keeps the file open and prints every new line as it arrives — the standard way to watch a log. Stop it with Ctrl+C.

Key in less	Action
`Space / b`	next / previous page
`↓ ↑`	one line
`/word`	search forward
`n  N`	next / previous match
`g  G`	start / end of file
`q`	quit

less logs/access.log · man pages use the same keys

LIVE DEMO

wc Counts, file Identifies

```
student@okan-lab:~/lab$ wc docs/linux_quotes.txt
  5  50 302 docs/linux_quotes.txt
student@okan-lab:~/lab$ wc -l data/*.csv data/*.txt
  41 data/students.csv
  30 data/cities.txt
  20 data/numbers.txt
  91 total
student@okan-lab:~/lab$ file README.txt src/hello.c data/students.csv photos/img…
README.txt:         ASCII text
src/hello.c:        C source, ASCII text
data/students.csv: CSV ASCII text
photos/img001.jpg: empty
projects:           directory
/usr/bin/ls:        ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dyna…
```

What to observe

- **wc:** lines, words, bytes — here 5, 50, 302.
- **-l -w -c:** only lines, words or bytes.
- students.csv: 41 lines = 1 header + 40 students.
- **file** reads the content (magic number, Week 7), not the extension: img001.jpg is "empty".

stat: Read the Inode

```
student@okan-lab:~/lab$ stat docs/syllabus.txt
  File: docs/syllabus.txt
  Size: 578             Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 804123      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1010/ student)   Gid: ( 1012/ student)
Access: 2026-10-12 08:47:57.251900492 +0000
Modify: 2026-10-12 08:47:57.254669759 +0000
Change: 2026-10-12 08:47:57.254669759 +0000
 Birth: 2026-10-12 08:47:57.251900492 +0000
```

```
student@okan-lab:~/lab$ stat -c "%n: %s bytes, inode %i, %h link(s)" docs/*.txt
docs/final report.txt: 53 bytes, inode 804125, 1 link(s)
docs/linux_quotes.txt: 302 bytes, inode 804124, 1 link(s)
docs/notes_01.txt: 16 bytes, inode 804127, 1 link(s)
...
```

Reading it

- **Size 578:** bytes of data.
- **Blocks 8:** 8 × 512 B = 4096 B = one 4 KiB block.
- **Inode, Links:** number and link count.
- **Access (0664/...):** permissions → Week 11.
- **-c:** print only chosen fields (%s %i %h ...).

LIVE DEMO

du and df: How Much Space?

```
student@okan-lab:~/lab$ du -sh docs data logs photos
32K     docs
16K     data
36K     logs
4.0K    photos
student@okan-lab:~/lab$ du -h projects
8.0K    projects/beta/src
8.0K    projects/beta/docs
24K     projects/beta
8.0K    projects/alpha/src
8.0K    projects/alpha/docs
24K     projects/alpha
52K     projects
student@okan-lab:~/lab$ df -h ~
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda        252G   13G   30G  29% /
```

What to observe

- **du:** disk space used by files, in whole blocks. -s total per argument, -h K/M/G.
- **photos 4.0K:** 15 empty files use 0 blocks; only the directory's 4 KiB block counts.
- **logs 36K** = 4 K (directory) + 24 K + 8 K (two logs).
- **df:** size, used and free space per file system. Use% = Used / (Used + Avail) ≈ 12 / 42 ≈ 29 %.

Hard Link, Copy and Symbolic Link Side by Side

```
student@okan-lab:~/lab/docs$ ln syllabus.txt plan.txt
student@okan-lab:~/lab/docs$ cp syllabus.txt copy.txt
student@okan-lab:~/lab/docs$ ln -s syllabus.txt shortcut.txt
student@okan-lab:~/lab/docs$ ls -li syllabus.txt plan.txt copy.txt shortcut.txt
804702 -rw-rw-r-- 1 student student 578 Oct 12 08:47 copy.txt
804123 -rw-rw-r-- 2 student student 578 Oct 12 08:47 plan.txt
804703 lrwxrwxrwx 1 student student  12 Oct 12 08:47 shortcut.txt -> syllabus.txt
804123 -rw-rw-r-- 2 student student 578 Oct 12 08:47 syllabus.txt
student@okan-lab:~/lab/docs$ echo "Week 16  Party" >> plan.txt
student@okan-lab:~/lab/docs$ tail -n 1 syllabus.txt copy.txt
==> syllabus.txt <==
Week 16  Party

==> copy.txt <==
Week 15 Final exam
student@okan-lab:~/lab/docs$ rm syllabus.txt
student@okan-lab:~/lab/docs$ ls -li plan.txt shortcut.txt
804123 -rw-rw-r-- 1 student student 593 Oct 12 08:47 plan.txt
804703 lrwxrwxrwx 1 student student  12 Oct 12 08:47 shortcut.txt -> syllabus.txt
student@okan-lab:~/lab/docs$ cat shortcut.txt
cat: shortcut.txt: No such file or directory
```

What to observe

- plan.txt = same inode 804123, link count 2.
- A change through plan.txt is visible in syllabus.txt, not in copy.txt.
- shortcut.txt: own inode, size 12 = length of "syllabus.txt".
- After rm: plan.txt keeps the data (count 1); shortcut.txt dangles.

Hard Link vs. Symbolic Link vs. Copy

	Hard link ln	Symbolic link ln -s	Copy cp
Inode	same as the target	its own (stores a path)	its own
Data	shared	reached through the path	duplicated
Target deleted	data stays (count − 1)	link dangles	copy unaffected
Directories	not allowed	allowed	with -r
Other file system	not possible	possible	possible

```
student@okan-lab:~/lab$ ln -s projects/alpha/docs alpha_docs
student@okan-lab:~/lab$ ls -l alpha_docs
lrwxrwxrwx 1 student student 19 Oct 12 08:47 alpha_docs -> projects/alpha/docs
student@okan-lab:~/lab$ ln projects alpha_hard
ln: projects: hard link not allowed for directory
student@okan-lab:~/lab$ ls -ld projects projects/alpha
drwxrwxr-x 4 student student 4096 Oct 12 08:47 projects
drwxrwxr-x 4 student student 4096 Oct 12 08:47 projects/alpha
```

Directory link count

2 + number of subdirectories: its name in the parent, its own . entry, and the .. of each child. projects: 2 + 2 = 4.

The Same Operations as System Calls in C

```c
CHECK(mkdir("box", 0775));                                  /* mkdir box         */
CHECK(fd = open("box/a.txt", O_WRONLY | O_CREAT, 0664)); /* touch box/a.txt   */
CHECK(write(fd, "hello\n", 6));  CHECK(close(fd));
CHECK(link("box/a.txt", "box/b.txt"));                      /* ln a.txt b.txt    */
CHECK(symlink("a.txt", "box/s.txt"));                       /* ln -s a.txt s.txt */
CHECK(rename("box/a.txt", "box/c.txt"));                    /* mv a.txt c.txt    */
CHECK(unlink("box/b.txt"));                                 /* rm b.txt          */
if (rmdir("box") == -1) perror("rmdir(\"box\")");           /* not empty: fails  */
CHECK(unlink("box/c.txt"));  CHECK(unlink("box/s.txt")); /* rm c.txt s.txt    */
CHECK(rmdir("box"));                                        /* rmdir box: now OK */
```

What to observe

- link(): same inode, links 2.
- rename(): inode unchanged.
- rmdir() fails while box is not empty.

```
$ ./fileops
after open(O_CREAT):
  box/a.txt  inode 804677  links 1
after link() and symlink():
  box/a.txt  inode 804677  links 2
  box/b.txt  inode 804677  links 2
  box/s.txt  inode 804678  links 1
after rename() a.txt -> c.txt:
  box/c.txt  inode 804677  links 2
after unlink() b.txt:
  box/c.txt  inode 804677  links 1
rmdir("box"): Directory not empty
box removed
```

CHEAT SHEET 2

Viewing, Inspecting and Linking

Command	What it does	Useful options
cat file...	print (concatenate) files	-n number lines
less file	page through a file	Space /word n q
head / tail file	first / last lines	-n N how many tail -f follow
wc file	count lines, words, bytes	-l -w -c
file name	identify the content type	—
stat file	show the inode	-c "%n %s %i %h" chosen fields
du / df	space used by files / free per file system	-s summary -h human-readable
ln / ln -s	hard link / symbolic link	ls -li to check inode and count

IN-CLASS LAB · 35 MINUTES

Lab Sheet: Part A (Basic) and Part B (Intermediate)

Fresh start: bash ~/mcmp117_lab_setup.sh && cd ~/lab — then do the exercises in order. Full text and points in the course notes.

Part A — 33 points

- 1 mkdir -p practice/a/b/c
- 2 three empty files with one touch
- 3 copy syllabus.txt into backup/ with a new name
- 4 rename it; prove the inode stays
- 5 first 4 and last 2 lines of the syllabus
- 6 count lines/words/bytes; how many students?
- 7 show "final report.txt" in two ways

Part B — 39 points

- 8 list photos with ? and [ ]
- 9 sort photos/ into jpg/, png/, other/
- 10 why does docs/* not show .settings?
- 11 back up docs/ with cp -r; compare with du
- 12 file types and disk usage
- 13 set a timestamp with touch -d; read it with stat

Lab Sheet: Part C (Challenge) and Grading

Part C needs the theory of Week 7: inodes, link counts, dangling links.

Part C — 28 points

- 14 hard link to the syllabus; delete the original; show the data survives
- 15 symbolic link alpha_docs; rename its target; repair the dangling link
- 16 one command: course/week01 … week14, each with slides/ and notes/ — how many directories?
- 17 clean up: why does rmdir practice fail? remove practice and course safely

How it is graded

- Total 100 points (A 33, B 39, C 28).
- Write the commands AND the key output lines.
- Explain in one sentence where asked ("why").
- Solutions with the expected output: last section of the notes.

Error Messages and What They Mean

Message (real)	Cause	Fix
cp: -r not specified; omitting directory	copying a directory	cp -r
rm: cannot remove 'x': Is a directory	rm without -r	rmdir (empty) or rm -r
rmdir: failed to remove 'x': Directory not empty	rmdir only removes empty dirs	empty it, or rm -r carefully
mkdir: cannot create directory 'a/b': No such file or directory	parent a missing	mkdir -p a/b
cat: docs/final: No such file or directory	unquoted space in a name	"docs/final report.txt"
ls: cannot access 'junk*': No such file or directory	the pattern matched nothing	check with ls / echo
ln: projects: hard link not allowed for directory	hard link to a directory	ln -s

QUICK QUIZ

Check Your Understanding

1. In ~/lab/photos, what does echo img01?.* print?

2. mv report.txt old/ or cp report.txt old/: which creates a new inode? Which is instant for a 4 GB file?

3. How many directories does mkdir -p site/{css,js,img}/{v1,v2} create, including site?

4. After ln a.txt b.txt; ln -s a.txt c.txt; rm a.txt — what do cat b.txt and cat c.txt print?

Week 10 — Key Takeaways

- **Names vs. inodes:** cp makes a new inode, mv renames (same inode), ln adds a name, rm removes a name — data is freed when the link count reaches 0.
- **Create:** mkdir -p for whole paths; touch for empty files and timestamps (atime, mtime, ctime).
- **Copy, move:** cp -r for directories; cp and mv overwrite silently — use -i.
- **Delete:** rm has no undo; rmdir only empty directories; look with ls before rm -r.
- **Expansion:** the shell expands * ? [ ] (existing names) and { } (generated text) before the command runs; echo shows the result.
- **Viewing:** cat, less, head, tail -f, wc, file, stat, du, df.
- **Every command is a system call:** mkdir(), renameat2(), linkat(), symlinkat(), unlinkat(), rmdir().

NEXT WEEK

Permissions, Ownership & Users

Week 11 will cover:

- Why protection? Links to dual mode (Week 2) and file protection (Week 7)
- Users and groups: id, whoami, /etc/passwd, /etc/group
- Reading -rw-rw-r--; chmod symbolic (u+x) and octal (754)
- umask, chown/chgrp, sudo — and Permission denied as student
- Lab: secure a project folder, make a script executable

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz